



Projetos de Algoritmos e Técnicas de Programação

Prof. Dr. Paulino Wagner Palheta Viana

Manaus, Abril 2026

Vetores

- É uma variável que pode armazenar vários valores.

meuVetor

--	--	--	--	--	--	--	--

medias

10	40	8	26	70	73		
----	----	---	----	----	----	--	--

nomes

Paulo	José	Maria	Ricardo				
-------	------	-------	---------	--	--	--	--

Vetores

- Os vetores são definidos pelo tipo de dados que eles devem armazenar e a quantidade de posições
- Exemplo:
 - Vetor de 8 posições para armazenar números reais
 - Vetor de 40 posições para armazenar caracteres
- Os vetores são estruturas homogêneas.
 - Ex: um vetor de inteiros só armazena dados do tipo inteiro.

Vetores

- Declaração:

<nome_variável>: vetor [posInicial..posFinal] de <tipo>

- Exemplo:

Algoritmo “exemplo”

var

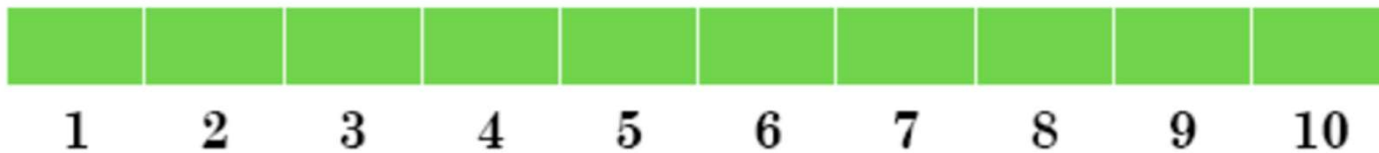
nome_alunos: vetor [1..50] de caractere

media_alunos: vetor [1..50] de real

inicio

Vetores

- Preenchendo e acessando um vetor
- As posições dos vetores são identificadas por índices
- Um vetor de 10 posições, por exemplo pode ser representado da seguinte forma:



Vetores

- Atribuição

`<nome_variável>[<poicao>]<- <valor>`

`Leia(<nome_variável> [<posição>])`

- Exemplo:

Algoritmo “exemplo”

var

nome_alunos: vetor [1..50] de caractere

media_alunos: vetor[1..50] de real

Inicio

nome_alunos[1]<- "Pedro"

Leia(nome_alunos[2])

media_alunos[1] <- 8.5

Vetores

- Preenchendo um vetor

- Podemos utilizar um laço de repetição para facilitar o preenchimento dos dados em vetores

- Exemplo:

Algoritmo “exemplo”

var

 numeros: vetor [1..50] de inteiro

 i: inteiro

Inicio

 para i de 1 ate 10 faca

 escreva(“Digite um valor “)

 leia(números[i])

 fimpara

fimalgoritmo

Vetores

- Exibindo o conteúdo de um vetor
 - Ou podemos utilizar um laço de repetição para facilitar a exibição dos valores de um vetor
- Exemplo:
 - para i de 1 ate 5 faca
 - escreva("O valor na posição ", i , "= ", numeros[i])
 - fimpara

Vetores

- Criar um algoritmo que leia 10 números pelo teclado e exiba os números na ordem correta que os números foram digitados.
- Exemplo:

```
algoritmo "vetor"  
var  
    numeros: vetor [1..10] de inteiro  
    i: inteiro  
inicio  
    para i de 1 ate 10 faca  
        escreva("Digite um número ")  
        leia(numeros[i])  
    fimpara  
    escreva("Numeros na ordem correta: ")  
    para i de 1 ate 10 faca  
        escreva(numeros[i])  
    fimpara  
fimalgoritmo
```

Vetores

- Criar um algoritmo que leia 10 números pelo teclado e exiba os números na ordem inversa da que os números foram digitados.
- Exemplo:

```
algoritmo "vetor"  
var  
    numeros: vetor [1..10] de inteiro  
    i: inteiro  
inicio  
    para i de 1 ate 10 faca  
        escreva("Digite um número ")  
        leia(numeros[i])  
    fimpara  
    escreva("Numeros na ordem inversa: ")  
    para i de 10 ate 1 passo -1 faca  
        escreva(numeros[i])  
    fimpara  
finalgoritmo
```

Vetores

- Escreva um algoritmo que leia um vetor com 10 posições de números inteiros. Em seguida, receba um novo valor do usuário e verifique se este valor se encontra no vetor.

```
algoritmo "busca_valor"  
var  
    numeros: vetor [1..10] de inteiro  
    i, valor : inteiro  
    encontrou: logico  
inicio  
    para i de 1 ate 10 faca  
        escreva("Digite um número ")  
        leia(numeros[i])  
    fimpara  
  
    escreva("Digite um número para ser buscado no vetor: ")  
    leia(valor)  
  
    encontrou <- falso  
    para i de 1 ate 10 faca  
        se (numeros[i] = valor) entao  
            encontrou <- verdadeiro  
        fimse  
    fimpara  
    se encontrou entao  
        escreva("O valor se encontra no vetor")  
    senao  
        escreva("O valor não se encontra no vetor")  
    fimse  
finalgoritmo
```

Vetores

- As estruturas de repetição (tanto para, enquanto e repita) permitem o uso do comando INTERROMPA
- Esse comando causa a saída imediata do laço de repetição

```
escreva("Digite um número para ser buscado no vetor: ");  
leia(valor)
```

```
encontrou <- falso  
para i de 1 ate 10 faca  
    se (numeros[i] = valor) entao  
        encontrou <- verdadeiro  
        interrompa  
    fimse  
fimpara
```

```
se (encontrou = verdadeiro) entao  
    escreva("O valor se encontra no vetor")  
senao  
    escreva("O valor não se encontra no vetor")  
fimse
```

Ao encontrar esse comando, o algoritmo passa a execução para o próximo comando após o laço.



Exercícios em Vetores

- Crie um algoritmo que leia um vetor de 10 números inteiros. Em seguida, calcule e escreva o somatório dos valores deste vetor.
- Escreva um algoritmo que leia um vetor com 15 posições de números inteiros. Em seguida, escreva somente os números positivos que se encontram no vetor.
- Escreva um algoritmo que leia um vetor com 8 posições de números inteiros. Em seguida, leia um novo valor do usuário e verifique se valor se encontra no vetor. Se estiver, informe a posição desse elemento no vetor. Caso o elemento não esteja no vetor, apresente uma mensagem informando “O número não se encontra no vetor”.

Exercícios em Vetores

- Escreva um algoritmo que leia dois vetores de 10 posições e faça a soma dos elementos de mesmo índice, colocando o resultado em um terceiro vetor. Mostre o vetor resultante.

- Exemplo:

vetor1	7	4	9	15	20	2	1	4	0	30
vetor2	1	8	3	7	14	9	1	8	11	16
vetorResultado	8	12	12	22	34	11	2	12	11	46

- Crie um algoritmo que leia um vetor de 20 posições e informe:
 - ☐ Quantos números pares existem no vetor
 - ☐ Quantos números ímpares existem no vetor
 - ☐ Quantos números maiores do que 50
 - ☐ Quantos números menores do que 7

Matrizes

- O que é uma matriz?
 - Uma estrutura de dados que contém várias variáveis do mesmo tipo
- Qual a diferença de vetores para matrizes?
 - Vetores são, na verdade, matrizes de uma única dimensão:

Vetores

1	3	4	6
a	maria	jota	

Matrizes

1	3	M	J	K	1.1	7.5	9.2	8.8
40	4	G	A	C	9.0	1.3	5.5	7.9
6	12	L	Z	H				

Matrizes

- As matrizes são, comumente referenciadas através de suas dimensões (quantidade de linhas e colunas)
- A notação comum é: $M \times N$, onde
 - M é a dimensão vertical (quantidade de linhas)
 - N é dimensão horizontal (quantidade de colunas)

3x3

3x2

2x3

4x1

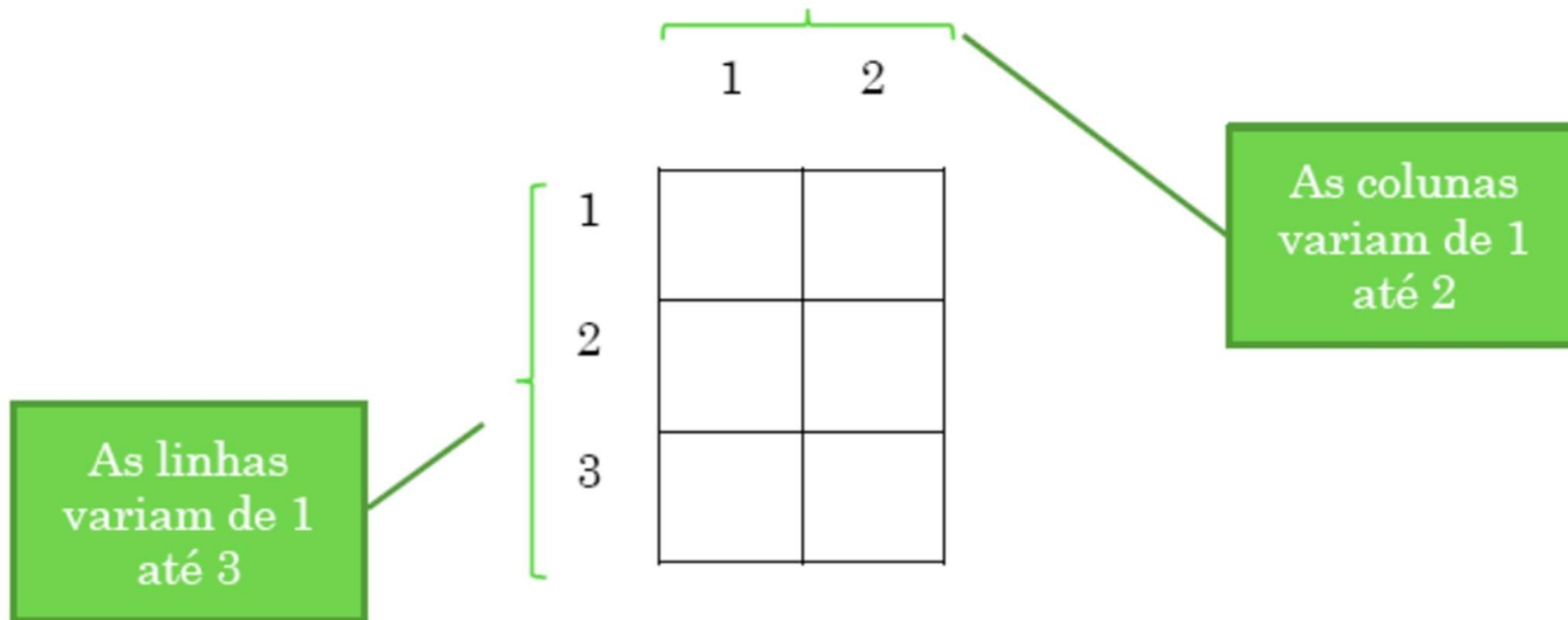
1x3

--	--	--

Vetores: a quantidade de linhas é sempre 1!

Matrizes

- Notação
- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (três linhas e duas colunas)



Matrizes

- Notação
- Como referenciar um elemento específico da matriz?
- Exemplo: Matriz 3x2 (três linhas e duas colunas)

	1	2
1	1,1	1,2
2	2,1	2,2
3	3,1	3,2

Diagram illustrating a 3x2 matrix with row and column indices. The matrix is shown with rows labeled 1, 2, and 3, and columns labeled 1 and 2. The elements are labeled as follows:

- Row 1: 1,1 (Column 1), 1,2 (Column 2)
- Row 2: 2,1 (Column 1), 2,2 (Column 2)
- Row 3: 3,1 (Column 1), 3,2 (Column 2)

Callouts highlight specific elements:

- Linha 3 Coluna 1 (Row 3, Column 1) points to the element 3,1.
- Linha 3 Coluna 2 (Row 3, Column 2) points to the element 3,2.

Matrizes

- Declaração:

<nome_variável>: vetor [li..lf, c1..cf] de <tipo>

- Onde:

- li e lf representam, respectivamente o índice inicial e final das linhas e
- ci e cf representam, respectivamente o índice inicial e final das colunas

Matrizes

■ Declaração:

<nome_variável>: vetor [li..lf, c1..cf] de <tipo>

■ Onde:

- li e lf representam, respectivamente o índice inicial e final das linhas e
- ci e cf representam, respectivamente o índice inicial e final das colunas

```
algoritmo "exemplo_matriz"  
var  
    exMatriz: vetor [1..3, 1..2] de inteiro  
inicio  
    ...
```

Linhas: o índice das
linhas varia de 1 até 3

Colunas: o índice das
colunas varia de 1 até 2

Matrizes

- Preenchendo e acessando uma matriz
 - As posições das matrizes são identificados pelos índices das linhas e colunas
- Atribuição.
`<nome_variável> [<linha>, <coluna>] <- <valor>`

Matrizes

■ Exemplo:

```
algoritmo "preencher"  
var  
    numeros: vetor[1..3, 1..2] de inteiro  
    i, j: inteiro  
inicio  
    escreva("Digite um valor para a posição [1,1]")  
    leia(numeros[1,1])  
    escreva("Digite um valor para a posição [1,2]")  
    leia(numeros[1,2])  
    escreva("Digite um valor para a posição [2,1]")  
    leia(numeros[2,1])  
    escreva("Digite um valor para a posição [2,2]")  
    leia(numeros[2,2])  
    escreva("Digite um valor para a posição [3,1]")  
    leia(numeros[3,1])  
    escreva("Digite um valor para a posição [3,2]")  
    leia(numeros[3,2])  
fimalgoritmo
```



Matrizes

- Preenchendo uma matriz
 - Entretanto, à medida que a quantidade de elementos da matriz aumenta, fica complicado fazermos manualmente para todas as posições.
- O melhor caminho é utilizar laços de repetição!

Matrizes

- Preenchendo uma matriz

- Entretanto, à medida que a quantidade de elementos da matriz aumenta, fica complicado fazermos manualmente para todas as posições.

- O melhor caminho é utilizar laços de repetição!

- Podemos criar um laço de repetição para variar pelas linhas, por exemplo:

```
algoritmo "preencher"  
var  
    numeros: vetor[1..3, 1..2] de inteiro  
    i, j: inteiro  
inicio  
    para i de 1 ate 3 faca  
        escreva("Digite um valor para a posição [", i, ",1]")  
        leia(numeros[i,1])  
        escreva("Digite um valor para a posição [", i, ",2]")  
        leia(numeros[i,2])  
    fimpara  
fimalgoritmo
```


Matrizes

■ Preenchendo uma matriz

- E podemos ainda incluir um laço de repetição para variar pelas colunas também, por exemplo:

```
algoritmo "preencher"  
var  
    numeros: vetor[1..3, 1..2] de inteiro  
    i, j: inteiro  
inicio  
    para i de 1 ate 3 faca  
        para j de 1 ate 2 faca  
            escreva("Digite um valor para a posição [", i, ",", j, "]: ")  
            leia(numeros[i,j])  
        fimpara  
    fimpara  
fimalgoritmo
```

Matrizes

- Exibindo o conteúdo de uma matriz
 - Ou podemos utilizar um laço de repetição para facilitar a exibição dos valores de uma matriz
 - Criando um laço para percorrer as linhas:
- Exemplo:

```
para i de 1 ate 3 faca
    escreval("O valor que está na posição [", i, ", 1] é: ", numeros[i, 1])
    escreval("O valor que está na posição [", i, ", 2] é: ", numeros[i, 2])
fimpara
```

Matrizes

- Criar um algoritmo que leia uma matriz 3x3 e exiba a matriz preenchida:

```
algoritmo "exemplo01"  
var  
    numeros: vetor[1..3, 1..3] de inteiro  
    i, j: inteiro  
inicio  
    para i de 1 ate 3 faca  
        para j de 1 ate 3 faca  
            escreva("Digite um valor para a posição [", i, ",", j, "]: ")  
            leia(numeros[i,j])  
        fimpara  
    fimpara  
  
    para i de 1 ate 3 faca  
        para j de 1 ate 3 faca  
            escreva(numeros[i, j])  
        fimpara  
    escreval  
    fimpara  
fimalgoritmo
```

Matrizes

- Criar um algoritmo que leia uma matrizes 3x3. Em seguida, exiba a soma dos elementos de cada uma das linhas. Ex:

1	2	2
3	2	3
4	1	1

Soma Linha 1 = 5

Soma Linha 2 = 8

Soma Linha 3 = 6

```
algoritmo "exemplo01"
var
    numeros: vetor[1..3, 1..3] de inteiro
    i, j: inteiro
    soma: inteiro
inicio
    para i de 1 ate 3 faca
        para j de 1 ate 3 faca
            escreva("Digite um valor para a posição [", i, ",", j, "]: ")
            leia(numeros[i,j])
        fimpara
    fimpara

    para i de 1 ate 3 faca
        soma <- 0
        para j de 1 ate 3 faca
            soma <- soma + numeros[i, j]
        fimpara
        escreval ("Soma Linha ", i, ": ", soma)
    fimpara
finalgoritmo
```



Matrizes

- Escreva um algoritmo que leia uma matriz 4x3. Em seguida, receba um novo valor do usuário e verifique se este valor se encontra na matriz. Caso o valor se encontre na matriz, escreva a mensagem “O valor se encontra na matriz”. Caso contrário, escreva a mensagem “O valor NÃO se encontra na matriz”.

Matrizes

```
algoritmo "exemplo03"
var
    numeros: vetor[1..4, 1..3] de inteiro
    i, j, buscar: inteiro
    achou: logico
inicio
    para i de 1 ate 4 faca
        para j de 1 ate 3 faca
            escreva("Digite um valor para a posição [", i, ",", j, "]: ")
            leia(numeros[i,j])
        fimpara
    fimpara

    escreva("Digite um valor para ser buscado na matriz: ")
    leia(buscar)

    achou <- falso
    para i de 1 ate 4 faca
        para j de 1 ate 3 faca
            se (numeros[i,j] = buscar) entao
                achou <- verdadeiro
            fimse
        fimpara
    fimpara
    se achou=verdadeiro entao
        escreva("O número se encontra na matriz.")
    senao
        escreva("O número NÃO se encontra na matriz.")
    fimse
fimalgoritmo
```

Exercícios de Matrizes

- Crie um algoritmo que leia uma matriz 5x5. Em seguida, conte quantos números pares existem na matriz.
- Crie um algoritmo que leia uma matriz 3x3 e calcule a soma dos valores das colunas da matriz. Ex:

1	2	2
3	2	3
4	1	1

Soma Coluna 1 = 8

Soma Coluna 2 = 5

Soma Coluna 3 = 6

Exercícios de Matrizes

- Crie um algoritmo que calcule a média dos elementos de uma matriz 5x2.
- 4.Crie um algoritmo informe qual o maior e qual o menor elemento existente em uma matriz 6x3.
- 5.Crie um algoritmo que leia uma matriz 3x3 e crie uma nova matriz que seja a matriz transposta da primeira (troque as linhas por colunas) Ex:

Matriz

1	2	3
4	5	6
7	8	9

Matriz Transposta

1	4	7
2	5	8
3	6	9

Exercícios de Matrizes

- Crie um algoritmo que calcule a média dos elementos de uma matriz 5x2.
- 4.Crie um algoritmo informe qual o maior e qual o menor elemento existente em uma matriz 6x3.
- 5.Crie um algoritmo que leia uma matriz 3x3 e crie uma nova matriz que seja a matriz transposta da primeira (troque as linhas por colunas) Ex:

Matriz

1	2	3
4	5	6
7	8	9

Matriz Transposta

1	4	7
2	5	8
3	6	9

Exercícios de Matrizes

- Crie um algoritmo que leia duas matrizes 2x5 e crie uma terceira matriz também 2x5 com o valor da soma dos elementos de mesmo índice. Ex:

Matriz1 + Matriz2 = Matriz3

1	2	2	4	3	6
3	2	5	3	8	5
4	1	7	7	11	8
5	5	4	4	9	9
1	2	1	9	2	11

Funções e Procedimentos

- Também chamados de subalgoritmos
- São trechos de algoritmos que efetuam um ou mais cálculos determinados
- Ao invés de escrever um código grande, pode-se escrever vários algoritmos menores (Modularização)
 - Em conjunto, resolvem o problema proposto
- É conveniente utilizá-los quando uma tarefa é efetuada em diversos lugares no mesmo algoritmo
- Ao invés de escrever um trecho diversas vezes, escreve-se um subalgoritmo e chama-o diversas vezes



Funções e Procedimentos

- Reduzem o tamanho do algoritmo
- Facilitam a compreensão e visualização do algoritmo
- São declarados no início do algoritmo e podem ser chamados em quaisquer pontos após sua declaração
- Podem ser:
 - Funções que retornam algum valor
 - Procedimento (ou subrotina) que não retorna nada



Funções e Procedimentos

- Uma função é um bloco de comandos que tem por objetivo retornar um valor ou uma informação
- A chamada de uma função é feita através da citação do seu nome seguido, opcionalmente, de seus argumentos iniciais entre parênteses
- As funções podem ser predefinidas pela linguagem ou criadas pelo programador

Funções e Procedimentos

- Funções predefinidas
- A linguagem do Visualg possui diversas funções predefinidas que podem ser usadas na construção de algoritmos
- Exemplo 1: Criar um algoritmo que calcule o valor da raiz quadrada de um número.
- O VisuAlg possui uma função predefinida que recebe como parâmetro uma variável (do tipo real) e retorna um valor (também do tipo real) referente à raiz quadrada do número.

Funções e Procedimentos

- Funções predefinidas

- Assinatura da função para cálculo da Raiz Quadrada

Raizq(valor : real) : real

Nome
da função

Parâmetro :
Tipo do parâmetro

Tipo
de retorno

Funções e Procedimentos

- Funções predefinidas
 - Como utilizar

```
algoritmo "Uso_de_Funcao_Predefinida"  
var  
    numero, raiz: real  
inicio  
    escreva("Digite um número: ")  
    leia(numero)  
    raiz <- Raizq(numero)  
    escreva("A raiz quadrada do número digitado é: ", raiz)  
fimalgoritmo
```


Funções e Procedimentos

■ Funções predefinidas

- Exemplo 2: Criar um algoritmo que, dada uma palavra qualquer, informe ao usuário quantas letras essa palavra possui.
- O Visualg possui uma função predefinida que recebe como parâmetro uma variável (tipo caractere) e retorna um valor (inteiro) correspondente à quantidade de caracteres existentes.

Funções e Procedimentos

■ Funções predefinidas

- Exemplo 2: Criar um algoritmo que, dada uma palavra qualquer, informe ao usuário quantas letras essa palavra possui.
- O Visualg possui uma função predefinida que recebe como parâmetro uma variável (tipo caractere) e retorna um valor (inteiro) correspondente à quantidade de caracteres existentes.

```
algoritmo "Uso_de_Funcao_Predefinida"  
var  
    palavra: caractere  
    qtd: inteiro  
inicio  
    escreva("Digite uma palavra qualquer: ")  
    leia(palavra)  
    qtd <- Compr(palavra)  
    escreva("A palavra que você digitou possui ", qtd, " caracteres")  
fimalgoritmo
```

Funções e Procedimentos

- Lista das Funções Predefinidas do Visualg
- Funções numéricas, algébricas e trigonométricas

Abs(expressão) - Retorna o valor absoluto de uma expressão do tipo inteiro ou real. Equivale a $| \text{expressão} |$ na álgebra.

ArcCos(expressão) - Retorna o ângulo (em radianos) cujo co-seno é representado por expressão.

ArcSen(expressão) - Retorna o ângulo (em radianos) cujo seno é representado por expressão.

ArcTan(expressão) - Retorna o ângulo (em radianos) cuja tangente é representada por expressão.

Cos(expressão) - Retorna o co-seno do ângulo (em radianos) representado por expressão.

CoTan(expressão) - Retorna a co-tangente do ângulo (em radianos) representado por expressão.

Exp(base, expoente) - Retorna o valor de base elevado a expoente, sendo ambos expressões do tipo real.

GraupRad(expressão) - Retorna o valor em radianos correspondente ao valor em graus representado por expressão.

Int(expressão) - Retorna a parte inteira do valor representado por expressão.

Funções e Procedimentos

- Lista das Funções Predefinidas do Visualg
- Funções numéricas, algébricas e trigonométricas

Log(expressão) - Retorna o logaritmo na base 10 do valor representado por expressão. **LogN(expressão)** - Retorna o logaritmo neperiano (base e) do valor representado por expressão.

Pi - Retorna o valor 3.141592.

Quad(expressão) - Retorna quadrado do valor representado por expressão.

RadpGrau(expressão) - Retorna o valor em graus correspondente ao valor em radianos representado por expressão.

RaizQ(expressão) - Retorna a raiz quadrada do valor representado por expressão.

Rand - Retorna um número real gerado aleatoriamente, maior ou igual a zero e menor que um.

RandI(limite) - Retorna um número inteiro gerado aleatoriamente, maior ou igual a zero e menor que limite.

Sen(expressão) - Retorna o seno do ângulo (em radianos) representado por expressão.

Tan(expressão) - Retorna a tangente do ângulo (em radianos) representado por expressão.

Funções e Procedimentos

- Lista das Funções Predefinidas do Visualg
- Funções para manipular cadeias de caracteres

Asc (s : character) : Retorna um inteiro com o código ASCII do primeiro caracter da expressão.

Carac (c : inteiro) : Retorna o caracter cujo código ASCII corresponde à expressão.

Caracpnum (c : character) : Retorna o inteiro ou real representado pela expressão. Corresponde a StrToInt() ou StrToFloat() do Delphi, Val() do Basic ou Clipper, etc.

Compr (c : character) : Retorna um inteiro contendo o comprimento (quantidade de caracteres) da expressão.

Copia (c : character ; p, n : inteiro) : Retorna um valor do tipo character contendo uma cópia parcial da expressão, a partir do caracter p, contendo n caracteres. Os caracteres são numerados da esquerda para a direita, começando de 1. Corresponde a Copy() do Delphi, Mid\$() do Basic ou Substr() do Clipper.

Maiusc (c : character) : Retorna um valor character contendo a expressão em maiúsculas.

Minusc (c : character) : Retorna um valor character contendo a expressão em minúsculas.

Numpcarac (n : inteiro ou real) : Retorna um valor character contendo a representação de n como uma cadeia de caracteres. Corresponde a IntToStr() ou FloatToStr() do Delphi, Str() do Basic ou Clipper.

Pos (subc, c : character) : Retorna um inteiro que indica a posição em que a cadeia subc se encontra em c, ou zero se subc não estiver contida em c. Corresponde funcionalmente a Pos() do Delphi, Instr() do Basic ou At() do Clipper, embora a ordem dos parâmetros possa ser diferente em algumas destas linguagens.

Funções e Procedimentos

■ Lista das Funções Predefinidas do Visualg

FUNÇÃO	DESCRIÇÃO
Abs (valor : real) : real	Valor absoluto
Arccos (valor : real) : real	Arco cosseno
Arcsen (valor : real) : real	Arco seno
Arctan (valor : real) : real	Arco tangente
Asc (s : caracter) : inteiro	Retorna o código ASCII
Compr (c : caracter) : inteiro	Retorna a dimensão do caractere
Copia (c : caracter , posini, posfin : inteiro) : caracter	Copia um determinado trecho do caractere
Cos (valor : real) : real	Cosseno
Cotan (valor : real) : real	Co-tangente
Exp (<base>, <expoente>)	Potenciação
Grauprad (valor : real) : real	Converte grau para radiano
Int (valor : real) : inteiro	Converte o valor em inteiro
Log (valor : real) : real	Logaritmo de base 10
Logn (valor : real) : real	Logaritmo natural (ln)

Maiusc (c : caracter) : caracter	Converte em Maiúscula
Minusc (c : caracter) : caracter	Converte em Minúscula
Numpcarac (n : inteiro ou real) : caracter	Converte um numero inteiro ou real para caractere
Pi : real	Valor Pi
Pos (subc, c : caracter) : inteiro	Retorna a posição do caractere.
Quad (valor : real) : real	Elevado quadrado
Radpgrau (valor : real) : real	Converte Radiano para grau.
Raizq (valor : real) : real	Raiz quadrada
Rand : real	Gerador de números aleatórios entre 0 e 1
Randi (limite : inteiro) : inteiro	Gerador de números inteiros aleatórios com um limite determinado
Sen (valor : real) : real	Seno
Tan (valor : real) : real	Tangente

Funções e Procedimentos

■ Funções predefinidas

- Exemplo3: Criar um algoritmo que, dado o nome de uma pessoa (Nome + Sobrenome), retorna apenas o sobrenome. Ex: “Alba Lopes” , retorna somente “Lopes”

```
algoritmo "Retorna_Sobrenome"  
var  
    nomeCompleto, sobrenome: caractere  
    qtdCaracteres, localEspaco: inteiro  
inicio  
    nomeCompleto <- "Alba Lopes"  
    qtdCaracteres <- Compr(nomeCompleto)  
    localEspaco <- Pos(" ", nomeCompleto)  
    sobrenome <- Copia(nomeCompleto, localEspaco + 1, qtdCaracteres)  
    escreva("O sobrenome é: ", sobrenome)  
fimalgoritmo
```

Funções e Procedimentos

■ Exercícios

Construa um algoritmo que receba dois valores do usuário (**a** e **b**) e realize a subtração entre eles. Apresente sempre o resultado positivo do cálculo. Ex: Para **a** = 2 e **b** = 9, o resultado de **a** - **b** deverá ser 7 (e não -7).

Construa um algoritmo que receba o nome completo de uma pessoa e escreva esse nome todo com letras maiúsculas

Construa um algoritmo que, tendo como dados de entrada dois pontos quaisquer no plano, P(x1,y1) e P(x2,y2), escreva a distância entre eles. A fórmula que efetua tal cálculo é:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Construa um algoritmo que resolva uma equação de segundo grau. Receba os três coeficientes **a**, **b** e **c** e informe as raízes da equação.

Construa um algoritmo que receba o e-mail de uma pessoa e verifique se o e-mail é válido. Considere que o e-mail é válido se possuir um @ e, no mínimo, 5 caracteres.

Funções e Procedimentos

- A criação de uma função deve ser realizada dentro da seção de variáveis
- Esse tipo de subalgoritmo sempre retorna apenas um valor para o algoritmo que o chamou
- As funções possuem um tipo de retorno associado
- Uma função pode possuir 0, 1 ou mais parâmetros

- Sintaxe:

```
algoritmo <nome do algoritmo>
var
    <declaração das variáveis globais>
    <definição das funções>
inicio
    <lista de comandos>
finalgoritmo
```

Funções e Procedimentos

■ Variáveis Locais:

- Declaradas dentro dos subalgoritmos (funções ou procedimentos)
- Podem ser usadas APENAS dentro das funções
- O algoritmo que chamou a função/procedimento não tem acesso à estas funções

■ Variáveis Globais

- São variáveis declaradas na seção var do algoritmo. Qualquer função/procedimento pode alterar o valor ou utilizá-la durante o seu processamento.

Funções e Procedimentos

- Exemplo 1: Criar uma função para calcular o dobro de um número passado como parâmetro

```
algoritmo "FuncoesPersonalizadas"  
var  
    numero, resultado: real  
  
    funcao Dobro(valor: real) : real  
    var  
        total: real  
    inicio  
        total <- valor * 2  
        retorne total  
    fimfuncao  
  
    inicio  
  
        escreva("Digite um número para calcular o dobro: ")  
        leia (numero)  
        resultado <- Dobro(numero)  
        escreva("O dobro é: ", resultado)  
  
    fimalgoritmo
```

Funções e Procedimentos

```
algoritmo "FuncoesPersonalizadas"
var
    numero: real

    funcao Dobro(valor: real) : real
    var
        total: real
    inicio
        total <- valor * 2
        retorne total
    fimfuncao

    funcao Triplo(valor: real) : real
    var
        total: real
    inicio
        total <- valor * 3
        retorne total
    fimfuncao

inicio

    escreva("Digite um número: ")
    leia (numero)
    escreval("O dobro é: ", Dobro(numero) )
    escreval("O triplo é: ", Triplo(numero) )

finalgoritmo
```



Funções e Procedimentos

- As funções podem possuir mais de um parâmetro
 - Parâmetros de um mesmo tipo são separados por vírgula
 - Parâmetros de tipos diferentes são separados por ponto e vírgula
- Exemplo 2: Criar uma função que receba três valores reais como parâmetro e retorne a média desses valores

Funções e Procedimentos

```
algoritmo "FuncoesPersonalizadas"  
var  
    numero1, numero2, numero3, resultado: real  
  
    funcao Media(n1, n2, n3: real) : real  
    var  
        calculoMedia: real  
    inicio  
        calculoMedia <- (n1 + n2 + n3) / 3  
        retorne calculoMedia  
    fimfuncao  
  
    inicio  
  
        escreval("Digite 3 numeros: ")  
        leia (numero1)  
        leia (numero2)  
        leia (numero3)  
        resultado <- Media(numero1, numero2, numero3)  
        escreva("O valor da média é: ", resultado)  
  
    fimalgoritmo
```



Funções e Procedimentos

- Exemplo 3: Escreva uma função que recebe as 4 notas de um aluno por parâmetro e uma letra. Se a letra for A a função calcula a média aritmética das notas do aluno, se for P, a sua média ponderada (pesos: 2, 3, 4 e 6).

Funções e Procedimentos

```
algoritmo "FuncoesPersonalizadas"
var
    numero1, numero2, numero3, resultado: real

    funcao MediaAouP( n1, n2, n3, n4: real ; tipo: caractere ) : real
    var
        calculo_media: real
    inicio
        escolha tipo
        caso "A"
            calculo_media <- (n1 + n2 + n3 + n4)/ 4
        caso "p"
            calculo_media <- (n1*2 + n2*3 + n3*4 + n4*6)/ 15
        outrocaso
            calculo_media <- 0
        fimescolha

        retorne calculo_media
    fimfuncao

inicio

    resultado <- MediaAouP(5, 6, 7, 8, "A")
    escreval("O valor da média é: ", resultado)

finalgoritmo
```




Funções e Procedimentos

- As funções podem não possuir parâmetros.
- Exemplo 4: Crie uma função que leia um número não determinado de valores positivos e calcule a soma desses números. A função deve parar de ler números quando um número negativo for digitado e retornar a soma dos números lidos.

Funções e Procedimentos

```
algoritmo "FuncaoSomaVarios"  
var  
    total: inteiro  
  
    funcao SomaVarios : inteiro  
    var  
        numero, soma: inteiro  
    inicio  
        soma <- 0  
        escreva("Digite um número: ")  
        leia(numero)  
        enquanto (numero >= 0) faca  
            soma <- soma + numero  
            escreva("Digite um número: ")  
            leia(numero)  
        fimenquanto  
        retorne soma  
    fimfuncao  
inicio  
  
    total <- SomaVarios  
    escreva("O valor da soma é: ", total)  
  
fimalgoritmo
```

Funções e Procedimentos

- Em Visualg, os procedimentos diferem das funções apenas por não retornarem valor nenhum
- A sintaxe utilizada na criação de procedimentos é:

```
procedimento <nome do procedimento> (<parâmetros>)  
var  
    <declaração das variáveis locais>  
inicio  
    <lista de comandos>  
fimprocedimento
```

Funções e Procedimentos

- Exemplo 1: Crie um procedimento que receba um valor como parâmetro e escreva o dobro desse número.

```
algoritmo "Procedimento"  
var  
    numero: inteiro  
  
    procedimento Dobro(valor: inteiro)  
        var  
            total: inteiro  
        inicio  
            total <- valor * 2  
            escreva("O dobro é: ", total)  
        fimprocedimento  
  
    inicio  
        escreva("Digite um número: ")  
        leia(numero)  
        Dobro(numero)  
    finalgoritmo
```

Não possui tipo de retorno e não retorna nada. Apenas executa o que está na seção de comandos

Funções e Procedimentos

- Exemplo 2: Crie um procedimento que receba um número como parâmetro e escreva a tabuada desse número.

```
algoritmo "ProcedimentoTabuada"  
var  
    numero: inteiro  
  
    procedimento Tabuada(valor: inteiro)  
    var  
        i: inteiro  
    inicio  
        escreval("*** Tabuada de ", valor, " ***")  
        para i de 1 ate 10 faca  
            escreval(valor, " *", i, " = ", valor*i)  
        fimpara  
    fimprocedimento  
  
    inicio  
        escreva("Digite um número: ")  
        leia(numero)  
        Tabuada(numero)  
    fimalgoritmo
```

Funções e Procedimentos

- Exemplo 3: Crie um algoritmo que utilize o procedimento criado anteriormente, e escreva a tabuada dos números de 1 a 9:

```
...  
  
    fimprocedimento  
  
inicio  
    para numero de 1 ate 9 faca  
        Tabuada(numero)  
    fimpara  
fimalgoritmo
```

Funções e Procedimentos

- Exemplo 4: Crie um procedimento que, dado um número N por parâmetro, desenhe o seguinte padrão na tela:
- Por exemplo, para $N = 5$

```
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

Funções e Procedimentos

```
algoritmo "ProcedimentoDesenharPadrao"  
var  
    numero: inteiro  
  
    procedimento DesenharPadrao(n: inteiro)  
    var  
        i, j: inteiro  
    inicio  
  
        para i de 1 ate n faca  
            para j de 1 ate n faca  
                escreva ("* ")  
            fimpara  
            escreval  
        fimpara  
  
    fimprocedimento  
  
inicio  
    escreva("Digite um valor: ")  
    leia(numero)  
    DesenharPadrao(numero)  
fimalgoritmo
```


Funções e Procedimentos

- Exemplo 5: Crie um procedimento que, dado um número N por parâmetro, desenhe o seguinte padrão na tela:
- Por exemplo, para N = 5

```
1  *  *  *  *
*  2      *
*      3      *
*          4  *
*  *  *  *  5
```

Funções e Procedimentos

```
algoritmo "ProcedimentoDesenharPadrao"  
var  
    numero: inteiro  
procedimento DesenharPadrao(n: inteiro)  
var  
    i, j: inteiro  
inicio  
  
    para i de 1 ate n faca  
        para j de 1 ate n faca  
            se (i = j) entao  
                escreva(j)  
            senao  
                se (i = 1) ou (i = n) entao  
                    escreva(" *")  
                senao  
                    se (j = 1) ou (j = n) entao  
                        escreva(" *")  
                    senao  
                        escreva ("  ")  
                fimse  
            fimse  
        fimse  
    fimpara  
    escreval  
fimpara  
  
fimprocedimento
```

```
inicio  
    escreva("Digite um valor: ")  
    leia(numero)  
    DesenharPadrao(numero)  
finalgoritmo
```

Funções e Procedimentos

- Parâmetros são canais por onde os dados são transferidos pelo algoritmo chamador a um subalgoritmo.

- ☐ Parâmetros Formais são os nomes simbólicos usados na definição dos parâmetros de um subalgoritmo.

```
procedimento soma(a, b: inteiro)  
var  
    calculo: inteiro
```

Parâmetros Formais

- ☐ Parâmetros Reais são aqueles que substituem os parâmetros formais quando da chamada de um subalgoritmo.

```
inicio  
    x <- 10  
    y <- 2  
    soma(x, y)  
fimalgoritmo
```

Parâmetros Reais



Funções e Procedimentos

- A substituição dos parâmetros formais pelos parâmetros reais no ato da invocação de um subalgoritmo é denominada de passagem de parâmetros
 - Pode se dar por dois mecanismos distintos:
 - Passagem por valor (ou por cópia)
 - Passagem por referência
- Na passagem por valor, é criada uma cópia dos parâmetros reais
 - As modificações efetuadas no parâmetro formal não afetam o parâmetro real, pois trabalha-se apenas com uma cópia.

Funções e Procedimentos

```
algoritmo "PassagemPorValor"  
var  
  x, y: inteiro  
  
procedimento soma(a, b: inteiro)  
  var  
    calculo: inteiro  
  inicio  
    calculo <- a + b  
    escreval("O valor da soma é ", calculo)  
    a <- 3  
    b <- 4  
  fimprocedimento  
  
inicio  
  x <- 1  
  y <- 2  
  soma(x, y)  
  escreval("Os valores de x e y são: ", x, y)  
fimalgoritmo
```

RESULTADO

```
O valor da soma é 3  
Os valores de x e y são: 1 2  
*** Fim da execução.
```

Mesmo alterando os valores de a e b, os valores de x e y continuam os mesmos após a chamada do subalgoritmo

Funções e Procedimentos

- O espaço de memória ocupado pelos parâmetros reais é compartilhado pelos parâmetros formais correspondentes
 - As modificações efetuadas nos parâmetros formais também afetarão os parâmetros reais
 - Na linguagem do Visualg, utiliza-se a palavra var antes do nome do parâmetro na declaração da função para informar que a passagem será por referência

Funções e Procedimentos

algoritmo "PassagemPorReferencia"

var

x, y: inteiro

procedimento soma(var a, b: inteiro)

var

calculo: inteiro

inicio

calculo <- a + b

escreval("O valor da soma é ", calculo)

a <- 3

b <- 4

fimprocedimento

inicio

x <- 1

y <- 2

soma(x, y)

escreval("Os valores de x e y são: ", x, y)

fimalgoritmo

A palavra var antes dos parâmetros indica passagem por referência

Funções e Procedimentos

- Exemplo 1: Crie um procedimento que receba dois valores inteiros por parâmetro e realize a troca desses valores.

```
algoritmo "TrocarValores"  
var  
    x, y: inteiro  
  
    procedimento troca(var a, b: inteiro)  
    var  
        auxiliar: inteiro  
    inicio  
        auxiliar <- a  
        a <- b  
        b <- auxiliar  
    fimprocedimento  
  
    inicio  
        x <- 5  
        y <- 8  
        escreval("Os valores de x e y ANTES da troca são: ", x, y)  
        troca(x, y)  
        escreval("Os valores de x e y DEPOIS da troca são: ", x, y)  
    fimalgoritmo
```


Funções e Procedimentos

- Um mesmo subalgoritmo pode ter parâmetros que são passados por valor e outros que são passados por referência:

Os parâmetros a e b são passados por valor e o parâmetro valorMedia é passado por referência

```
algoritmo "ValorERreferencia"
var
    n1, n2, resultado: real

procedimento media(a, b: real; var valorMedia: real)
var
    soma: real
inicio
    soma <- a + b
    valorMedia <- soma/2
fimprocedimento

inicio
    n1 <- 5
    n2 <- 2
    media(n1, n2, resultado)
    escreva("O valor da média é: ", resultado)
fimalgoritmo
```



Funções e Procedimentos

- As variáveis locais são visíveis apenas dentro dos subalgoritmo que as criou
 - Já as variáveis globais são visíveis tanto no algoritmo principal como nos subalgoritmos
 - Por esse motivo, é importante que as funções/procedimentos sejam declarados na seção var após a declaração das variáveis globais
 - É importante não criar variáveis locais e globais com o mesmo nome para evitar ambiguidades.

Funções e Procedimentos

```
algoritmo "variaveisLocaisEGlobais"  
var  
    resultado: inteiro  
  
procedimento soma(a, b: inteiro)  
var  
    calculo: inteiro  
inicio  
    calculo <- a + b  
    resultado <- calculo  
fimprocedimento  
  
inicio  
    soma(2, 3)  
    escreva("O valor da soma é: ", resultado)  
  
fimalgoritmo
```



Funções e Procedimentos

- O Visualg ainda não permite a passagem de Vetores e Matrizes como parâmetros de subalgoritmos
 - Para criar subalgoritmos que precisam de vetores e matrizes, utilize variáveis globais
 - Exemplo 1: Crie um procedimento para preencher um vetor de posições

Funções e Procedimentos

```
algoritmo "vetorEmSubalgoritmos"  
var  
    numeros: vetor [1..10] de inteiro  
  
    procedimento preencherVetor  
        var  
            i: inteiro  
        inicio  
            para i de 1 ate 10 faca  
                escreva("Digite o valor para a posição ", i, " do vetor: ")  
                leia(numeros[i])  
            fimpara  
        fimprocedimento  
  
    inicio  
        preencherVetor  
    fimalgoritmo
```

Funções e Procedimentos

- Exemplo 2: Crie um procedimento para exibir o conteúdo de um vetor

```
algoritmo "AlgImprimirVetor"  
var  
    numeros: vetor [1..10] de inteiro  
    cont: inteiro  
    procedimento ImprimirVetor  
        var  
            i: inteiro  
        inicio  
            para i de 1 ate 10 faca  
                escreva(numeros[i])  
            fimpara  
        fimprocedimento  
  
inicio  
    para cont de 1 ate 10 faca  
        numeros[cont] <- cont  
    fimpara  
    ImprimirVetor  
fimalgoritmo
```

Funções e Procedimentos

- Exemplo 3: Crie um procedimento para imprimir o conteúdo de um vetor de 10 posições. Esse procedimento deve receber um parâmetro do tipo caractere que indica se o conteúdo deve ser exibido na ordem correta ou na ordem inversa (“C” para correta e “I” para inversa)

Funções e Procedimentos

```
algoritmo "AlgImprimirVetor"  
var  
  numeros: vetor [1..10] de inteiro  
  cont: inteiro  
  
procedimento ImprimirVetor(ordem: caractere)  
var  
  i: inteiro  
inicio  
  
  se (ordem = "C") entao  
    para i de 1 ate 10 faca  
      escreva(numeros[i])  
    fimpara  
  senao  
    se (ordem = "I") entao  
      para i de 10 ate 1 passo -1 faca  
        escreva(numeros[i])  
      fimpara  
    fimse  
  fimse  
  
fimprocedimento  
  
inicio  
  para cont de 1 ate 10 faca  
    numeros[cont] <- cont  
  fimpara  
  ImprimirVetor ("I")  
  ImprimirVetor ("C")  
fimalgoritmo
```




Projetos de Algoritmos e Técnicas de Programação

Prof. Dr. Paulino Wagner Palheta Viana

Manaus, Abril 2026